
AI Resume Ranker using Machine Learning and NLP

AI & Machine Learning Internship Project Report

Submitted By

N. Karthik

Internship

Elevate Labs – AI & Machine Learning Internship

Abstract

The AI Resume Ranker is a web-based application developed using Python, Flask, and Machine Learning techniques to simplify the recruitment process. The application compares multiple resumes with a given Job Description (JD) and ranks candidates according to their relevance. It extracts important candidate information such as name, email, phone number, and skills while calculating a similarity score using Natural Language Processing (NLP). The system generates a ranked report, helping recruiters identify the most suitable candidates quickly and accurately.

1. Introduction

Recruitment is a time-consuming process when recruiters manually screen hundreds of resumes. Many resumes contain different formats and writing styles, making evaluation difficult.

The AI Resume Ranker automates this process using Natural Language Processing (NLP) and Machine Learning. It extracts text from PDF resumes, compares resume content with a job description, calculates similarity scores, and ranks candidates based on their suitability.

2. Problem Statement

Manual resume screening has several disadvantages:

- Time-consuming
- Human bias
- Difficulty handling large numbers of resumes

- Inconsistent evaluation

The proposed system addresses these issues by automatically analyzing resumes and ranking candidates based on objective similarity scores.

3. Objectives

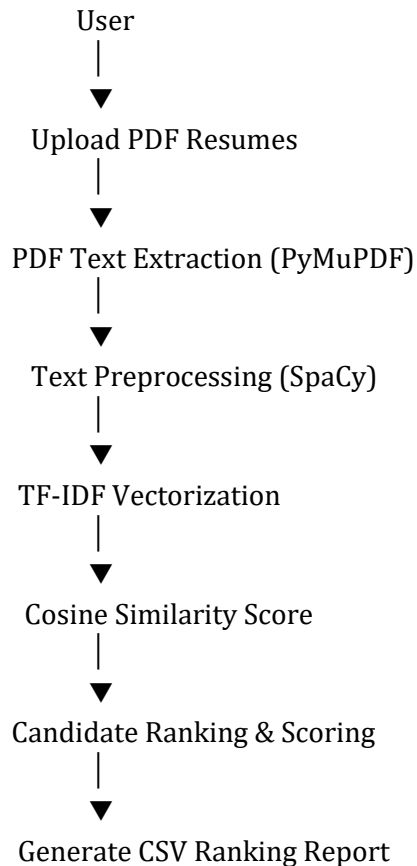
- Automate resume screening.
- Compare resumes with job descriptions.
- Extract candidate information.
- Match candidate skills with required skills.
- Rank resumes according to relevance.
- Generate downloadable reports.

4. Technologies Used

Technology Purpose

Python	Programming Language
Flask	Backend Web Framework
HTML	User Interface
CSS	Styling
Scikit-learn	Machine Learning
SpaCy	Natural Language Processing
Pandas	Data Processing
PyMuPDF	PDF Text Extraction
CSV	Report Generation

5. System Architecture



6. Methodology

Step 1: Resume Upload

Users upload one or more resumes in PDF format.

Step 2: Job Description Input

Recruiters enter the required Job Description.

Step 3: Text Extraction

The application extracts text from each resume using PyMuPDF.

Step 4: NLP Processing

SpaCy cleans and preprocesses the extracted text by removing unnecessary words and formatting.

Step 5: Feature Extraction

TF-IDF Vectorization converts textual information into numerical vectors.

Step 6: Similarity Calculation

Cosine Similarity compares each resume with the job description.

Step 7: Ranking

Candidates are ranked based on similarity scores.

Step 8: Report Generation

A CSV report is generated containing rankings and candidate details.

7. Features

- Upload multiple PDF resumes
 - Enter Job Description
 - Resume ranking
 - Candidate name extraction
 - Email extraction
 - Phone number extraction
 - Skill matching
 - Missing skills identification
 - Resume recommendations
 - CSV report generation
-

8. Project Structure

```
Ai-Resume_Ranker/  
|  
├─ app.py  
├─ utils.py  
├─ requirements.txt  
├─ skills.txt  
|  
├─ templates/  
|   ├─ index.html  
|   └─ results.html
```

```
|
|
├── static/
│   └── style.css
|
├── uploads/
├── reports/
└── README.md
```

9. Algorithms Used

TF-IDF (Term Frequency–Inverse Document Frequency)

TF-IDF measures the importance of words in a resume relative to the job description.

Cosine Similarity

Cosine Similarity calculates the similarity between the resume vector and the job description vector.

Similarity values range from:

- **100%** – Perfect Match
 - **70–99%** – Excellent Match
 - **50–69%** – Good Match
 - **Below 50%** – Needs Improvement
-

10. Workflow

1. Upload resumes.
2. Enter Job Description.
3. Extract resume text.
4. Clean text using NLP.
5. Generate TF-IDF vectors.
6. Compute cosine similarity.
7. Rank resumes.

8. Display candidate information.
 9. Download CSV report.
-

11. Results

The system successfully:

- Ranked candidates based on similarity.
 - Extracted candidate details automatically.
 - Displayed matched and missing skills.
 - Generated downloadable ranking reports.
 - Reduced manual screening effort.
-

12. Advantages

- Saves recruiter time.
 - Improves hiring accuracy.
 - Handles multiple resumes efficiently.
 - Generates objective rankings.
 - Easy-to-use web interface.
-

13. Limitations

- Supports PDF resumes only.
 - Ranking depends on the quality of the job description.
 - Uses content-based matching and may not capture every nuance of candidate experience.
-

14. Future Enhancements

- Support DOCX resumes.

- AI-based resume feedback.
 - Resume keyword suggestions.
 - ATS integration.
 - Dashboard with analytics.
 - Cloud database support.
 - User authentication.
 - Email notification system.
-

15. Conclusion

The AI Resume Ranker demonstrates how Machine Learning and Natural Language Processing can automate resume screening and improve recruitment efficiency. By extracting resume information, comparing it with job descriptions, and ranking candidates objectively, the application reduces manual effort and supports better hiring decisions. The project successfully showcases the practical application of AI in human resource management.